# ASSIGNMENT – 1

**Name:** M. Pavan Kumar

**Reg No:** 1925260235

**Course Name:** Data Structures

**Course Code:** CSA0305

**Slot:** B

# 1. What is the sizeof Operator and Its Importance?

## Definition

The **sizeof** operator in C is used to find the amount of memory, in bytes, occupied by a data type, variable, or expression. Its result is of type **size_t**. The exact size of some data types can differ depending on the compiler and computer system, so sizeof is useful when we need the actual size on the system where the program is running.

## Syntax

```
sizeof(data_type)
sizeof(variable)
```

## Example

```
int age = 18;
float mark = 92.5;
char grade = 'A';

printf("%zu", sizeof(age));
printf("%zu", sizeof(mark));
printf("%zu", sizeof(grade));
```

On many common systems, an int uses 4 bytes, a float uses 4 bytes, and a char uses 1 byte. These are typical results, not guaranteed values for every system. Running the program shows the sizes used by the current compiler.

## Importance of sizeof

1. **Finding memory usage:** It tells us how much space a variable or data type occupies, which helps us understand a program's memory requirements.
2. **Working with arrays:** For an array such as int numbers[5], sizeof(numbers) gives the total size of the whole array in bytes. If one int occupies 4 bytes, the array typically occupies 20 bytes.
3. **Dynamic memory allocation:** It helps calculate the number of bytes to request using functions such as malloc. For example, malloc(5 * sizeof(int)) requests enough space for five int values.
4. **Writing portable code:** Instead of assuming that every data type has a particular size, a program can ask the compiler for the size.
5. **Avoiding memory mistakes:** Correct size calculations can help prevent allocating too little or too much memory.

## Conclusion

The sizeof operator is a simple but important C feature. It measures memory size in bytes and is especially useful with arrays, structures, and dynamic memory allocation. It reports the size used by the current system rather than a universal fixed value.

# 2. Compare Static and Dynamic Memory Allocation with Examples

## Meaning

**Static memory allocation** means memory size is decided before or at the start of program execution and is not resized automatically during execution. For example, int marks[5]; creates an array with space for five integers. Its capacity is fixed.

**Dynamic memory allocation** means memory is requested while the program is running. In C, functions such as malloc(), calloc(), realloc(), and free() are used for this purpose. The program can request a size based on its current needs and release the memory when it is no longer required.

## Differences

1. **Size:** Static allocation has a fixed size; dynamic allocation can be requested according to runtime needs.
2. **Time:** Static storage is arranged according to the declaration and storage duration; dynamic memory is requested during execution.
3. **Flexibility:** A fixed array cannot grow beyond its declared capacity. Dynamically allocated memory can be resized using realloc, if resizing succeeds.
4. **Management:** The programmer generally does not manually release an ordinary local array. Dynamically allocated memory must be released using free() when it is no longer needed.
5. **Risk:** A fixed array may be too small or waste space. Dynamic allocation offers flexibility but can cause memory leaks or invalid memory access if handled incorrectly.

## Example of static allocation

```
int numbers[5];
numbers[0] = 10;
numbers[1] = 20;
```

This array can store five integers. Its capacity does not change while the program runs.

## Example of dynamic allocation

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *numbers = malloc(n * sizeof(int));
    if (numbers == NULL) return 1;
    numbers[0] = 10;
    numbers[1] = 20;
    free(numbers);
    return 0;
}
```

Here, malloc requests enough memory for five integers. The NULL check confirms that allocation succeeded before the memory is used. The free function releases the allocated block.

## Conclusion

Static allocation is simple when the required size is known. Dynamic allocation is useful when the amount of data may change or is not known in advance. A good programmer chooses the method that fits the problem and manages dynamically allocated memory carefully.

# 3. Solve String Reversal Using Recursion

## Definition of recursion

Recursion is a programming technique in which a function calls itself to solve a smaller version of the same problem. A recursive function must have a **base case** that stops further calls; otherwise, it may continue calling itself until the program runs out of stack space.

## Logic for reversing a string

To reverse a string recursively, we can print or process the last character first and then repeat the same operation for the remaining characters. The function below moves forward through the string during recursive calls and prints characters while the calls return. This prints the characters in reverse order.

## C Program

```
#include <stdio.h>

void reverseString(char str[], int index) {
    if (str[index] == '\0') {
        return; /* base case */
    }

    reverseString(str, index + 1);
    printf("%c", str[index]);
}

int main(void) {
    char str[] = "HELLO";
    printf("Original string: %s\n", str);
    printf("Reversed string: ");
    reverseString(str, 0);
    printf("\n");
    return 0;
}
```

## Output

```
Original string: HELLO
Reversed string: OLLEH
```

## Step-by-step explanation

**Step 1:** The function starts at index 0, which points to H.
**Step 2:** It calls itself with the next index until it reaches the null character '\0', which marks the end of the string.
**Step 3:** The calls then return in reverse order, so the function prints O, L, L, E, and H.
**Step 4:** The final output is OLLEH.

**Base case:** If str[index] is '\0', the function returns without making another call.
**Recursive case:** The function calls reverseString(str, index + 1) and then prints the current character.

## Complexity and conclusion

For a string of n characters, the function visits each character once, so its time complexity is **O(n)**. The recursive calls use stack space proportional to the string length, so the auxiliary space complexity is **O(n)**. Recursion makes the solution short and demonstrates how a problem can be solved through smaller steps, though a loop may use less stack memory for very long strings.